OpenClaw Robotic Arm Control

OpenClaw Robotic Arm Control

1. Course Content
    - Course Overview
    - Learning Objectives
2. Preparation
3. Robotic Arm-Related Control Interfaces
4. Case Demonstration
    - 4.1 Start OpenClaw Interaction
    - 4.2 Get the Current Joint Angles of the Robotic Arm
    - 4.3 Control the 6 Joints of the Robotic Arm Simultaneously
    - 4.4 Control a Single Robotic Arm Joint
    - 4.5 Restore the Robotic Arm to the Preset Initial Pose
    - 4.6 Adjust the Camera Observation View
5. Source Code Analysis
        - `ControlArmJointAngles()` — Control 6 Joints Simultaneously
        - `AdjustCameraView()` — Adjust the Camera View
        - `InitArmPose()` — Initialize the Robotic Arm Pose
        - `ControlSingleArmJointAngle()` — Control a Single Joint Angle

1. Course Content

Course Overview

- OpenClaw robotic arm control is one of the core features of the OpenClaw platform, converting AI natural language instructions into robotic arm control operations through the MCP protocol
- Covers multiple robotic arm control interfaces: joint angle retrieval, end-effector pose retrieval, simultaneous multi-joint control, single-joint control, camera view adjustment, and initial pose restoration

Learning Objectives

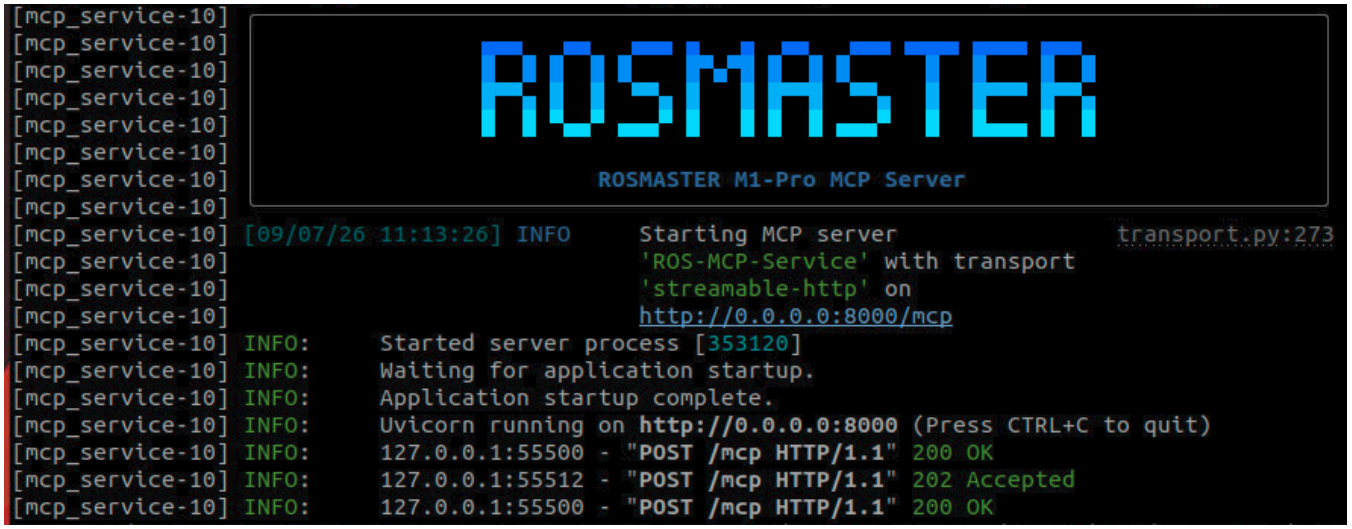
1. Understand the interface features of OpenClaw controlling the robotic arm
2. Master the practical operation methods and use cases of each robotic arm control interface
3. Be able to optimize grasping and placement accuracy by adjusting configuration file parameters

2. Preparation

- Start the MCP service

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```

- If you need voice replies, you need to add the use_tts parameter: `use_tts:=true`, otherwise it can be omitted



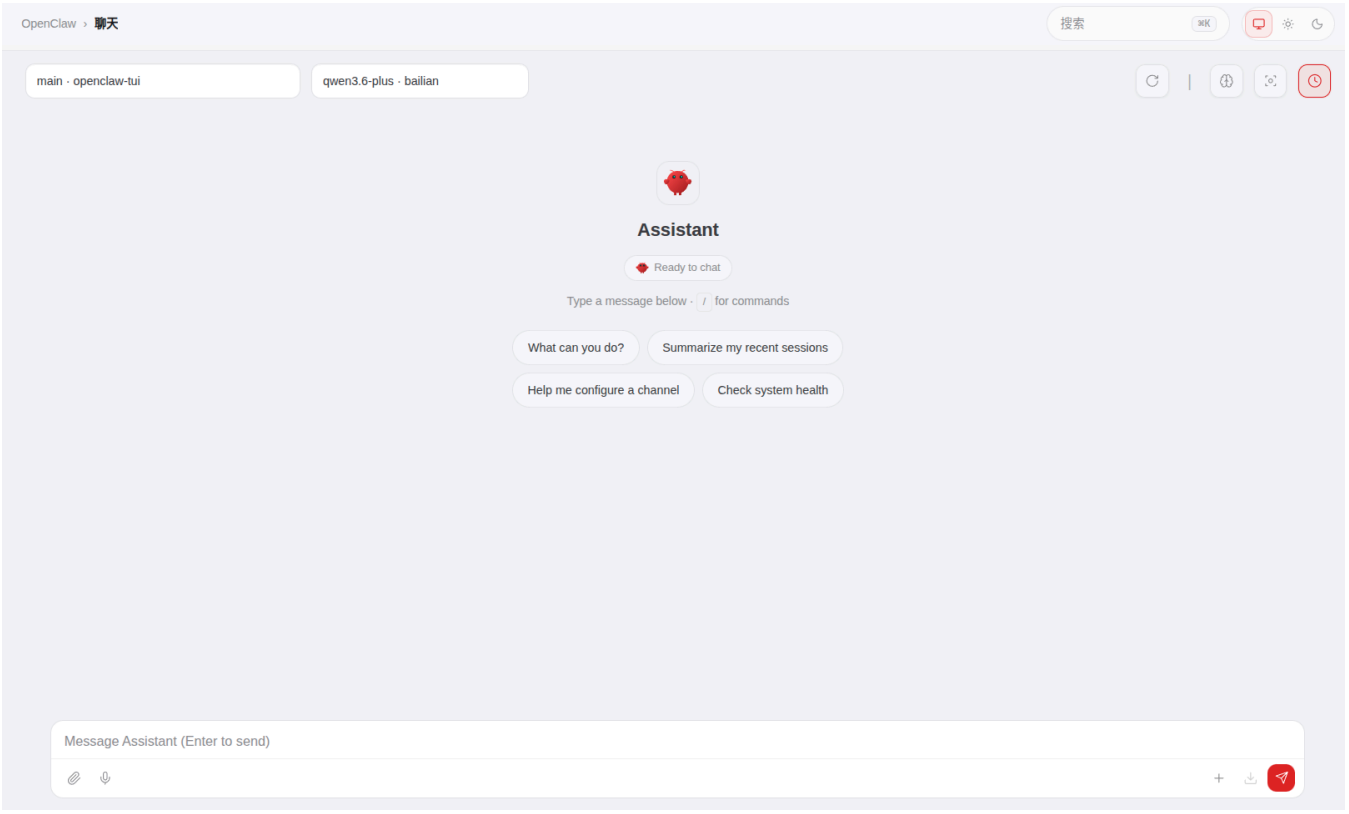
3. Robotic Arm-Related Control Interfaces

No.	Tool Name	Function Description
1	GetCurrentArmJointAngles	Get the current joint angles of the robotic arm
2	ControlSixArmJoint	Control the 6 joints of the robotic arm simultaneously
3	ControlSingleArmJoint	Control a single robotic arm joint to a specified angle
4	AdjustCameraView	Adjust the camera view
5	InitArmPose	Restore the robotic arm to the preset initial pose

4. Case Demonstration

4.1 Start OpenClaw Interaction

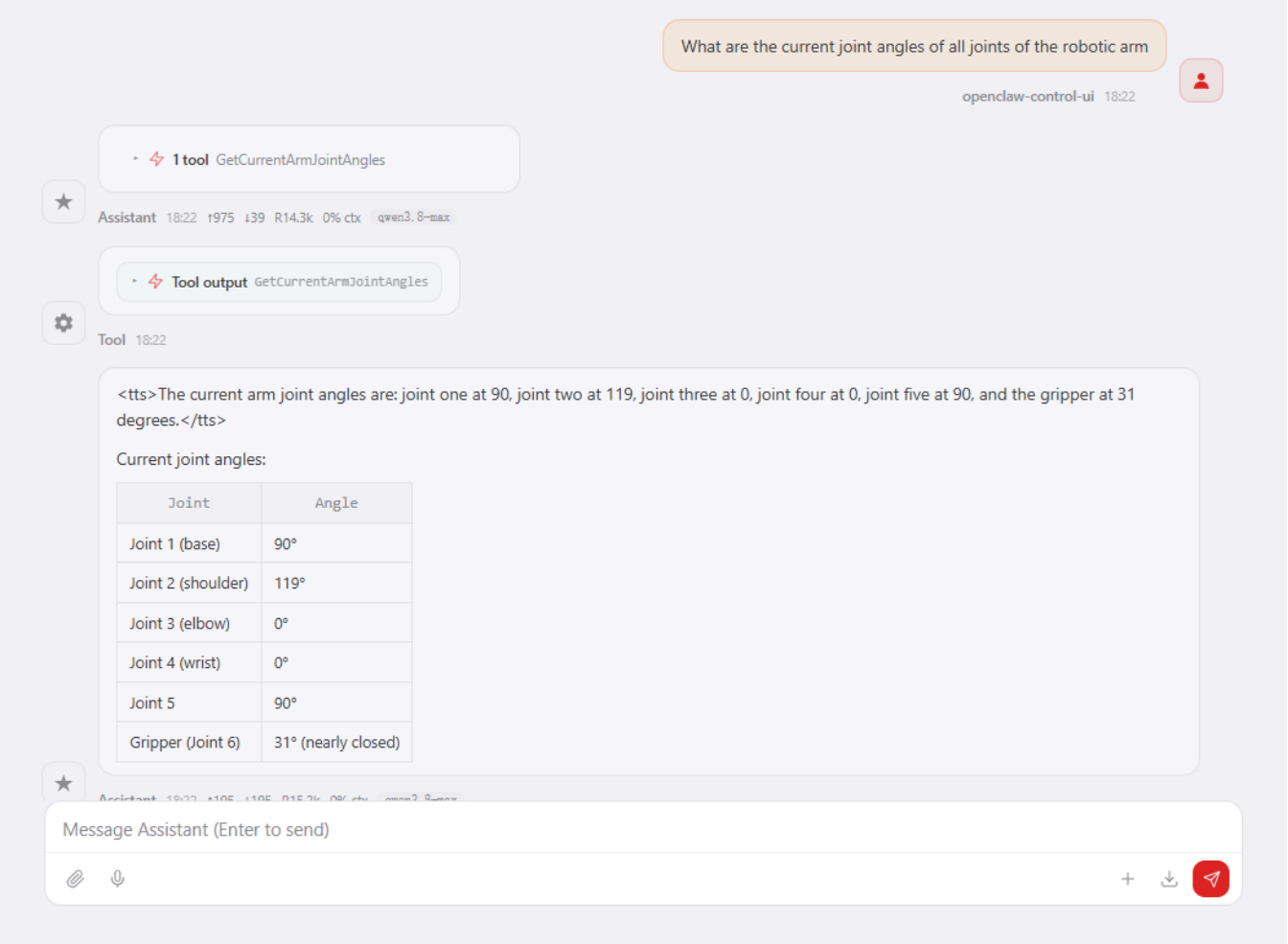
- Use any interaction method to converse with OpenClaw, here demonstrating with Web Chat



> **Tip**
>
> - You can write the commands according to your own needs; following is the case demonstration

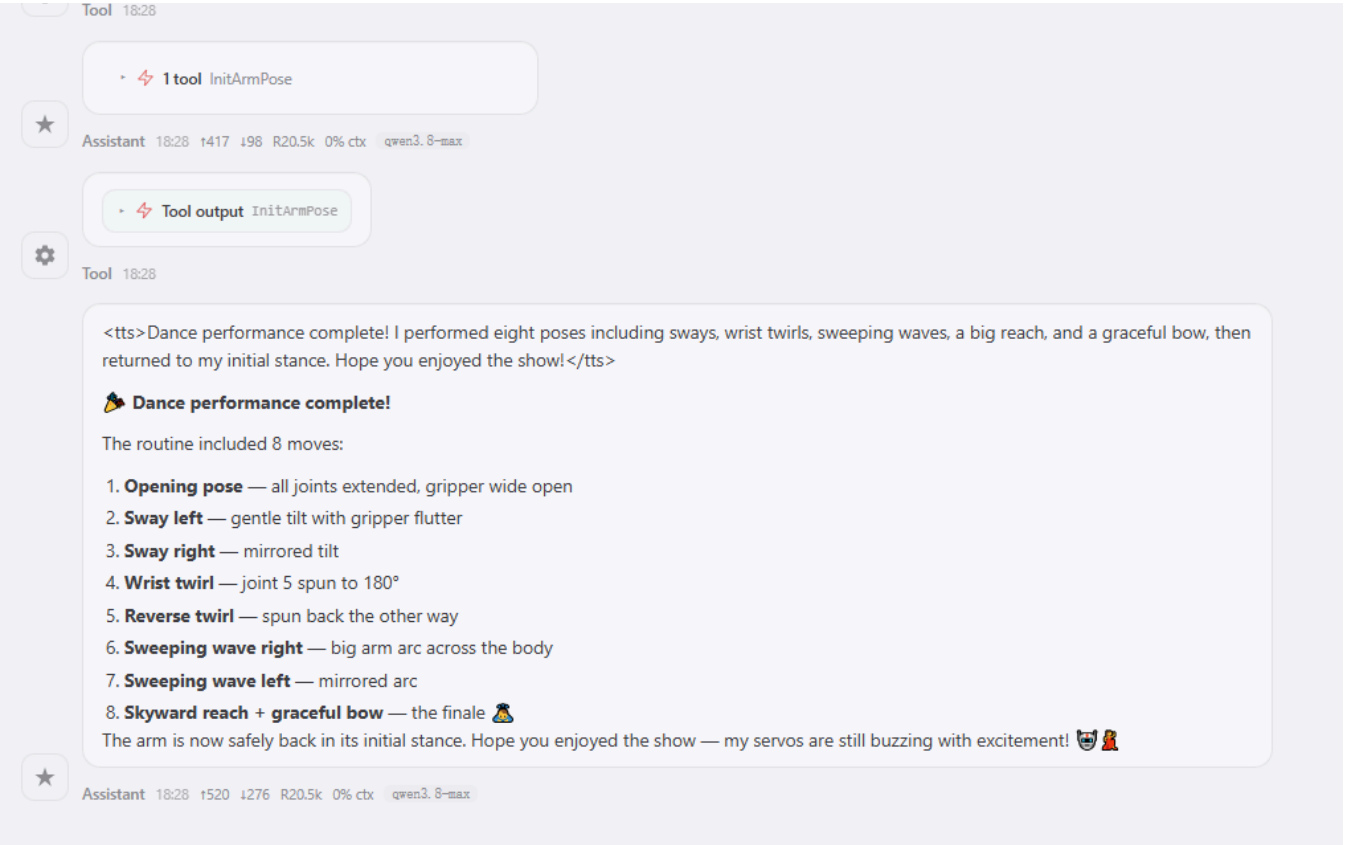
4.2 Get the Current Joint Angles of the Robotic Arm

- What are the current angles of each joint of the robotic arm?



4.3 Control the 6 Joints of the Robotic Arm Simultaneously

- Use the robotic arm to perform a dance for me
- OpenClaw autonomously decides and choreographs the robotic arm dance movements; the actual effect shall prevail, and different models output different content



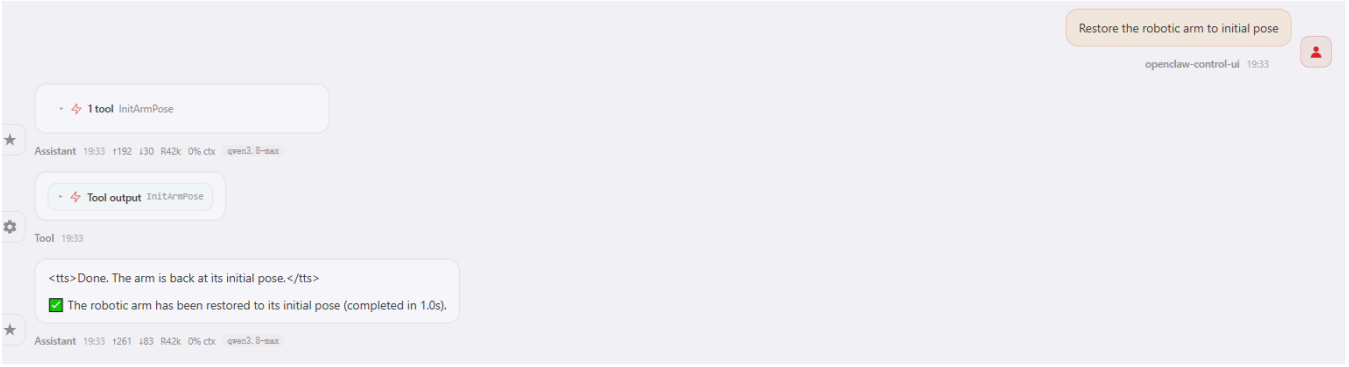
4.4 Control a Single Robotic Arm Joint

- Set the first joint of the robotic arm to 0 degrees



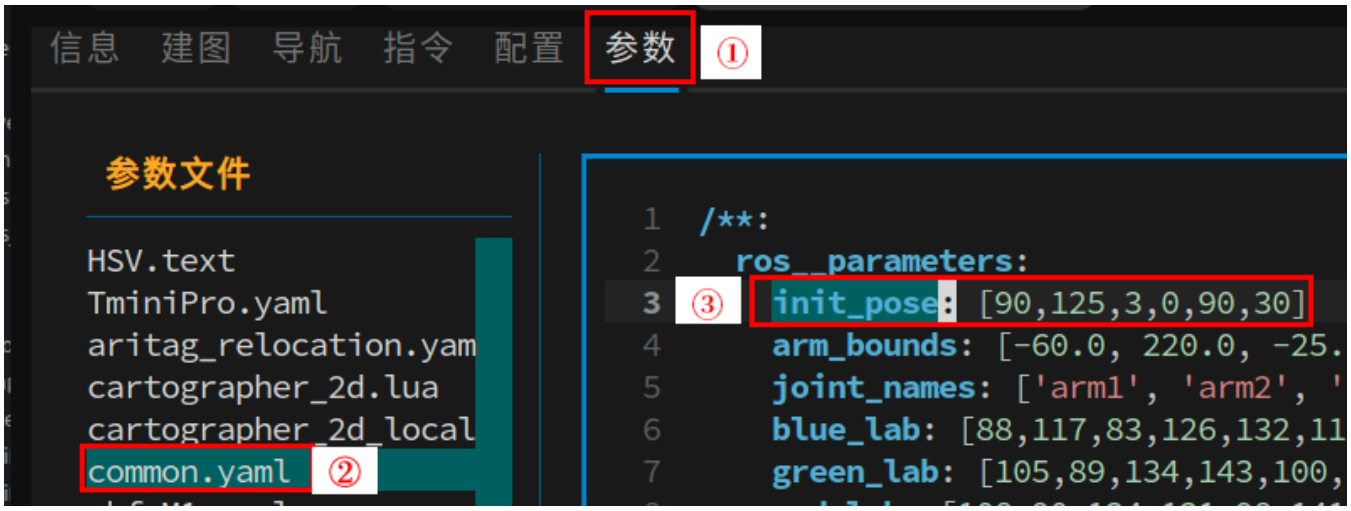
4.5 Restore the Robotic Arm to the Preset Initial Pose

- Restore the robotic arm to the initial pose



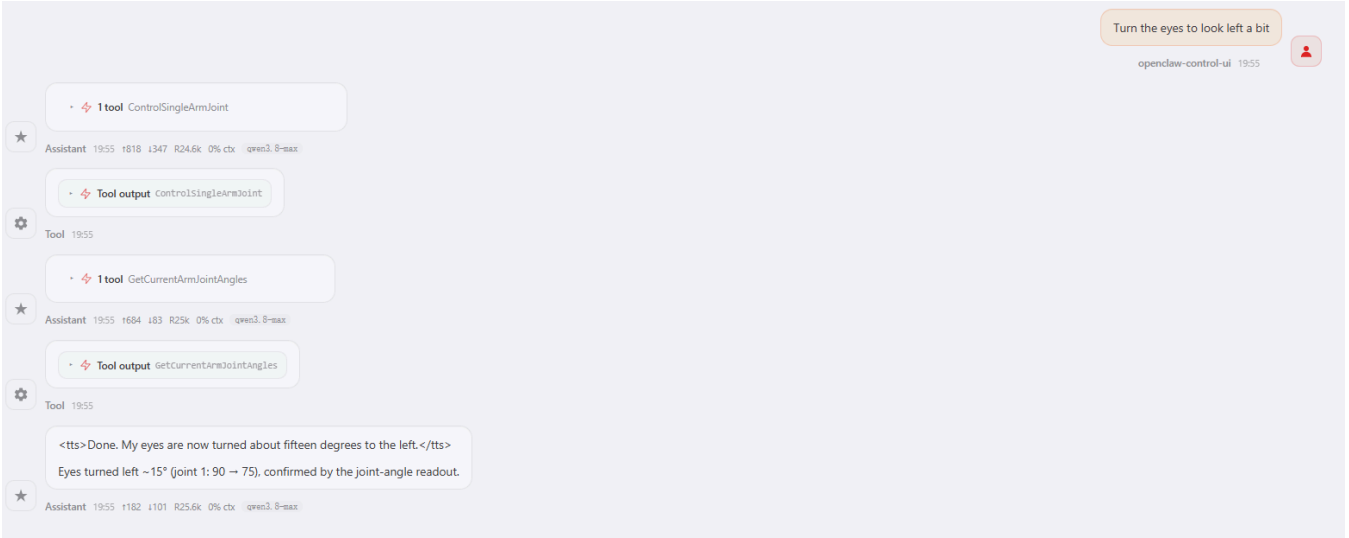
- The initial pose can be changed through the configuration file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```



4.6 Adjust the Camera Observation View

- Look a bit to the left
- This feature is used to conveniently adjust the observation view to find targets during subsequent robotic arm operations



5. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

`ControlArmJointAngles()` — Control 6 Joints Simultaneously

Code Description:

- Constructs an `ArmJoints` ROS2 message, setting the target angles of the 6 joints at once and publishing to the `arm6_joints` topic
- Joint 6 (the gripper) has an `180 - angle` conversion, because the gripper's mechanical zero point and the angle definition direction are opposite
- The `runtime` parameter controls the movement time (milliseconds); the larger the value, the slower and smoother the movement

```
def ControlArmJointAngles(self, arm_joint_1:int, arm_joint_2:int, arm_joint_3:int,
                          arm_joint_4:int, arm_joint_5:int, arm_joint_6:int,
                          runtime:int=2000) -> bool:
    arm_joint = ArmJoints()
    arm_joint.joint1 = arm_joint_1
    arm_joint.joint2 = arm_joint_2
    arm_joint.joint3 = arm_joint_3
    arm_joint.joint4 = arm_joint_4
    arm_joint.joint5 = arm_joint_5
    arm_joint.joint6 = 180 - arm_joint_6  # Special handling for joint 6
    arm_joint.time = runtime               # Movement time (ms)
    self.SixJoint_pub.publish(arm_joint)
    return True
```

`AdjustCameraView()` — Adjust the Camera View

Code Description:

- **yaw**: Adjusts joint 1 to rotate the camera left and right; positive values turn right, negative values turn left
- **pitch**: Preferentially adjusts joint 4 to achieve up/down tilt; when joint 4 reaches its lower limit (-5°), it automatically switches to joint 2 for pitch compensation
- After adjustment is complete, it automatically calls `SeeWhat()` to refresh the image and returns the new image path

```
def AdjustCameraView(self, pitch:int=0, yaw:int=0) -> list[bool, str]:
    """Quickly adjust the camera view"""
    self._get_joints()  # Get the current angles of joints 1, 2, and 4
```

```
    if yaw != 0:  # Horizontal adjustment (joint 1 - left/right rotation)
        self.joint_1 += yaw
        self.SingleJoint_pub.publish(ArmJoint(id=1, joint=self.joint_1, time=200))

    if pitch != 0:  # Pitch adjustment (joint 4 - up/down tilt)
        temp_joint = self.joint_4 + pitch
        if temp_joint <= -5:  # Joint 4 reached its lower limit; use joint 2 for pitch
compensation
            self.joint_2 += pitch
            self.SingleJoint_pub.publish(ArmJoint(id=2, joint=self.joint_2, time=200))
        else:
            self.SingleJoint_pub.publish(ArmJoint(id=4, joint=temp_joint, time=200))

    self._reset_joints()
    res = self.SeeWhat()  # Automatic image refresh
    return [True, res[1]] if res[0] else [False, res[1]]
```

`InitArmPose()` — Initialize the Robotic Arm Pose

Code Description:

- `InitArmPose`: Restores the robotic arm to the preset safe initial pose, avoiding the robotic arm being in a dangerous or interfering position
- `ArmJoints()`: Creates a 6-joint message object for simultaneously controlling all joints
- `self.init_arm_pose[0]~[5]`: The initial pose angle array read from the configuration file `common.yaml`, assigned to joint1~joint6 in sequence
- `arm_joint.time = 2000`: Sets the movement time to 2000 ms (2 seconds), ensuring the robotic arm transitions smoothly to the initial pose
- `self.SixJoint_pub.publish(arm_joint)`: Publishes the joint angles to the `arm6_joints` topic, driving the robotic arm
- `time.sleep(1.0)`: Blocks for 1 second, ensuring the robotic arm basically reaches the target position before returning

```
def InitArmPose(self) -> bool:
    '''Initialize the robotic arm to a safe pose.'''
    # Step 1: Create the joint message object
    arm_joint = ArmJoints()

    # Step 2: Set the target angles of the 6 joints
    arm_joint.joint1 = self.init_arm_pose[0]
    arm_joint.joint2 = self.init_arm_pose[1]
    arm_joint.joint3 = self.init_arm_pose[2]
    arm_joint.joint4 = self.init_arm_pose[3]
    arm_joint.joint5 = self.init_arm_pose[4]
    arm_joint.joint6 = self.init_arm_pose[5]

    # Step 3: Set the movement time (2000 ms = 2 seconds)
    arm_joint.time = 2000
```

```
    # Step 4: Publish the joint angles to the topic
    self.SixJoint_pub.publish(arm_joint)

    # Step 5: Wait for the robotic arm movement to complete
    time.sleep(1.0)
    return True
```

Function Description:

1. **Safe pose initialization**: Restores the robotic arm to the preset safe initial pose, avoiding the robotic arm being in a dangerous or interfering position
2. **6-joint synchronous control**: Simultaneously controls the 6 joints (joint1~joint6) of the robotic arm to move to the target angles
3. **Movement time control**: Sets the movement duration to 2 seconds through `time=2000`, ensuring the robotic arm transitions smoothly to the initial pose
4. **Blocking wait**: `time.sleep(1.0)` waits for the robotic arm to basically reach the target position before returning

---

`ControlSingleArmJointAngle()` — Control a Single Joint Angle

Code Description:

- `ControlSingleArmJointAngle`: Individually controls a certain joint of the robotic arm to a specified angle, without affecting the state of other joints
- `arm_joint_id == 6` branch: The 6th joint is the gripper, which performs an `angle = 180 - angle` flip, because the gripper's mechanical zero point and the angle definition direction are opposite; after the flip it matches intuitive control (180 = fully open, 0 = fully closed)
- `ArmJoint(id=arm_joint_id, joint=angle, time=runtime)`: Constructs a single-joint control message, where `id` is the joint number (1~6), `joint` is the target angle, and `time` is the movement time (milliseconds)
- `self.SingleJoint_pub.publish(...)`: Publishes the single-joint control command to the topic, driving the corresponding joint

```
def ControlSingleArmJointAngle(self, arm_joint_id: int, angle: int, runtime: int = 1500) ->
bool:
    """ Control a single joint of the robotic arm"""
    # Step 1: Special handling for joint 6 (the gripper)
    if arm_joint_id == 6:
        angle = 180 - angle  # Gripper angle flip

    # Step 2: Publish the single-joint control command
    self.SingleJoint_pub.publish(
        ArmJoint(id=arm_joint_id, joint=angle, time=runtime)
    )
    return True
```

Function Description:

1. **Independent single-joint control**: Allows controlling a single joint of the robotic arm independently, without affecting other joints
2. **Gripper angle flip**: Joint 6 (the gripper) has special handling `angle = 180 - angle`, because 0° and 180° of the gripper correspond to the closed and open states respectively, requiring an angle flip to match intuitive control
3. **Configurable movement time**: The `runtime` parameter defaults to 1500 ms (1.5 seconds) and can be adjusted as needed to change the movement speed

Parameter Description:

Parameter	Type	Description	Range
`arm_joint_id`	int	Joint ID (1~6)	-
`angle`	int	Target angle (degrees)	Depending on the joint type, the range is usually 0~180°
`runtime`	int	Movement time (milliseconds)	Default 1500, recommended range 500~3000